

# C++23 Ranges Library Expansion: The Maturity Update

A practical guide to C++23 Ranges features like zip, enumerate, chunk, and join\_with — replacing manual loops with modern pipelines

10 min read · 2 days ago



Sagar

Following ▾



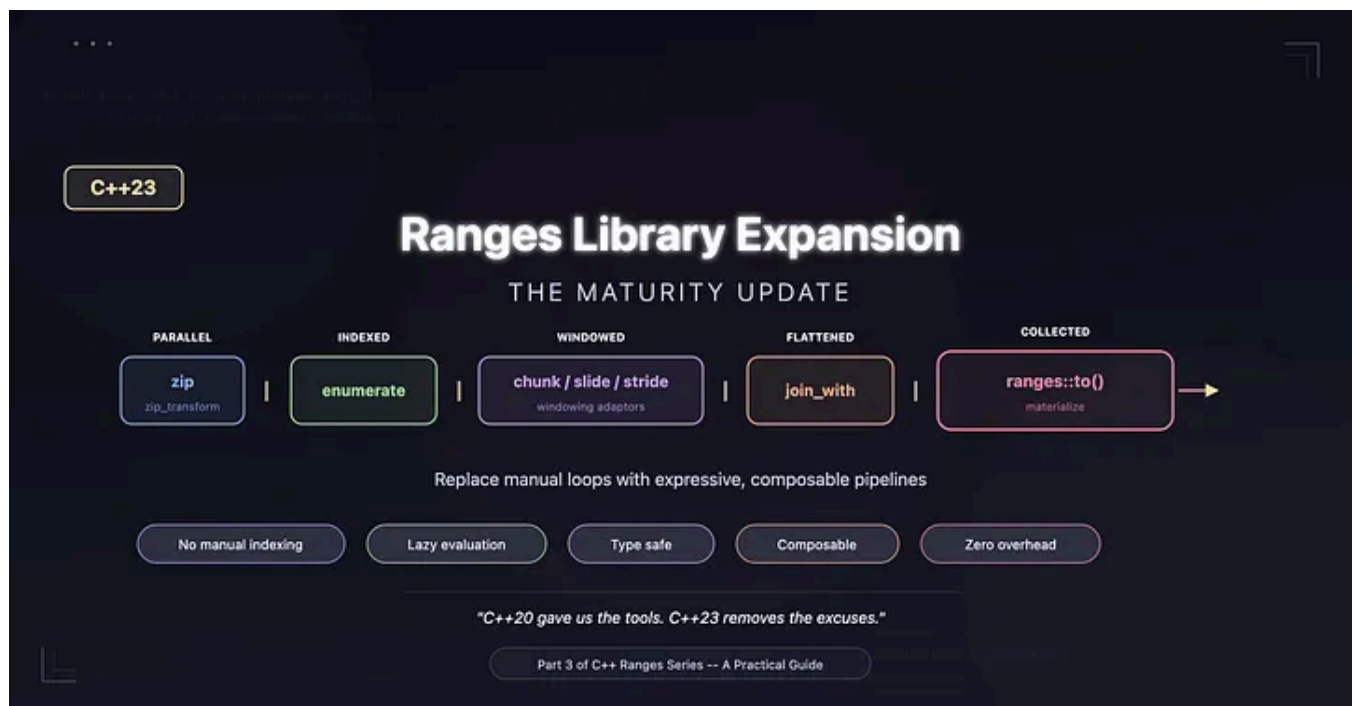
Listen



Share



More



## Prerequisites:

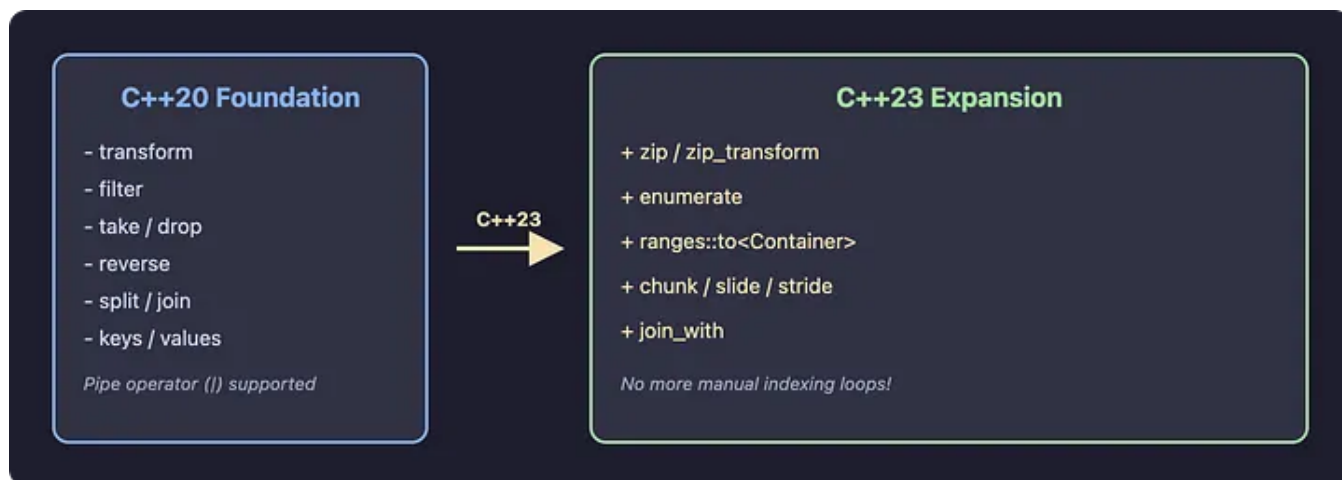
You should be comfortable with C++20 `std::ranges` and `std::views`. If not, check out my earlier articles:

[C++20 `std::views` Guide — Ranges Pipelines](#)

[C++20 Ranges Deep Dive — Core Mechanics](#)

C++20 gave us ranges and views and made ranges powerful. C++23 fills in the missing pieces that made us still reach for raw loops. C++23 makes them practical.

Here's the mental model:



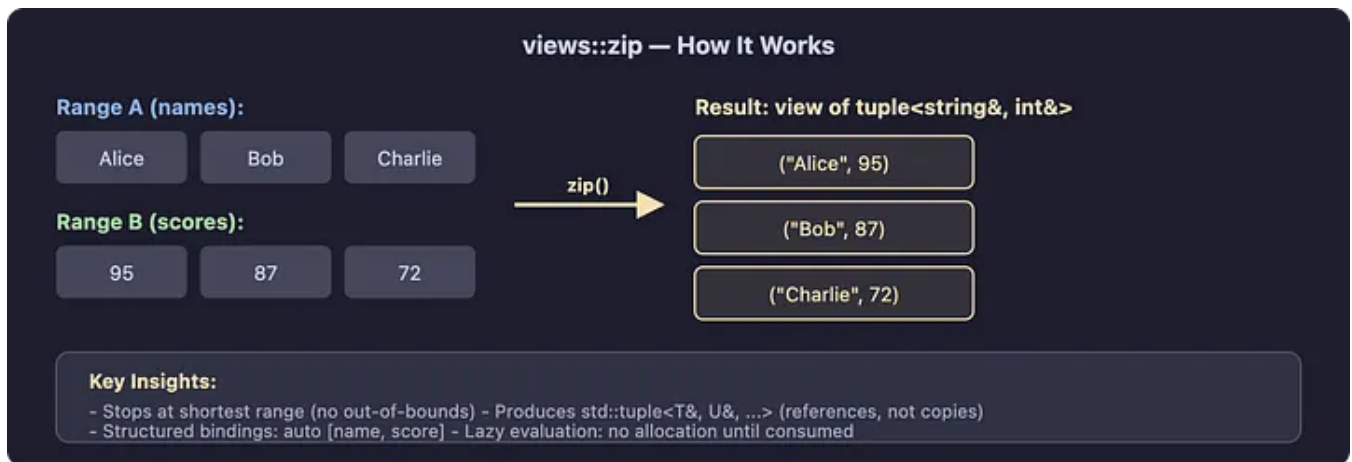
| C++23 Ranges — Pattern to Tool Map                                                                                                                          |            |                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|---------------------------------------------------------------|
| Match your problem to the right view adaptor                                                                                                                |            |                                                               |
| Pattern                                                                                                                                                     | C++23 Tool |                                                               |
|  <b>Parallel Iteration</b><br>Walk multiple containers side by side      | -----●     | <code>views::zip</code> / <code>views::zip_transform</code>   |
|  <b>Indexed Iteration</b><br>Need {index, value} pairs during loop       | -----●     | <code>views::enumerate</code>                                 |
|  <b>Materialization</b><br>Convert lazy pipeline into a real container   | -----●     | <code>ranges::to&lt;Container&gt;()</code>                    |
|  <b>Windowing / Batching</b><br>Group, slide, or skip through elements   | -----●     | <code>chunk</code> / <code>slide</code> / <code>stride</code> |
|  <b>Flatten + Delimiter</b><br>Join nested ranges with separator between | -----●     | <code>views::join_with</code>                                 |
| All adaptors are lazy. Materialize with <code>ranges::to&lt;&gt;</code> when you need a real container. Compile with <code>-std=c++23</code> .              |            |                                                               |

Let's walk through each one with real code.

## 1. `std::views::zip` & `zip_transform`

```
std::views::zip
```

`std::views::zip` takes two or more ranges and produces a single view where each element is a `std::tuple` of references — one from each input range. Think of it as "walking multiple containers side by side in one loop."



### Key Points:

- Stops at the shortest range. If `names` has 3 elements and `scores` has 5, you get 3 tuples. No out-of-bounds, no manual `std::min`.
- Produces `std::tuple<T&, U&, ...>` — these are *references*, not copies. Modifying through them modifies the originals.
- Structured bindings ( `auto [name, score]` ) work perfectly with the returned tuples.
- Lazy evaluation — no intermediate container is created. Elements are fetched on demand.
- Works with any number of ranges: `zip(r1, r2, r3, r4)` is valid.

### Pre-C++23: Manual index juggling

```
#include <vector>
#include <string>
#include <iostream>

int main() {
    std::vector<std::string> names = {"Alice", "Bob", "Charlie"};
    std::vector<int>          scores = {95, 87, 72};

    // Manually track index, pray both vectors are same size
```

```

auto size = std::min(names.size(), scores.size());
for (size_t i = 0; i < size; ++i) {
    std::cout << names[i] << " scored " << scores[i] << "\n";
}
}

```

## C++23: Clean and safe

```

#include <ranges>
#include <vector>
#include <string>
#include <print>

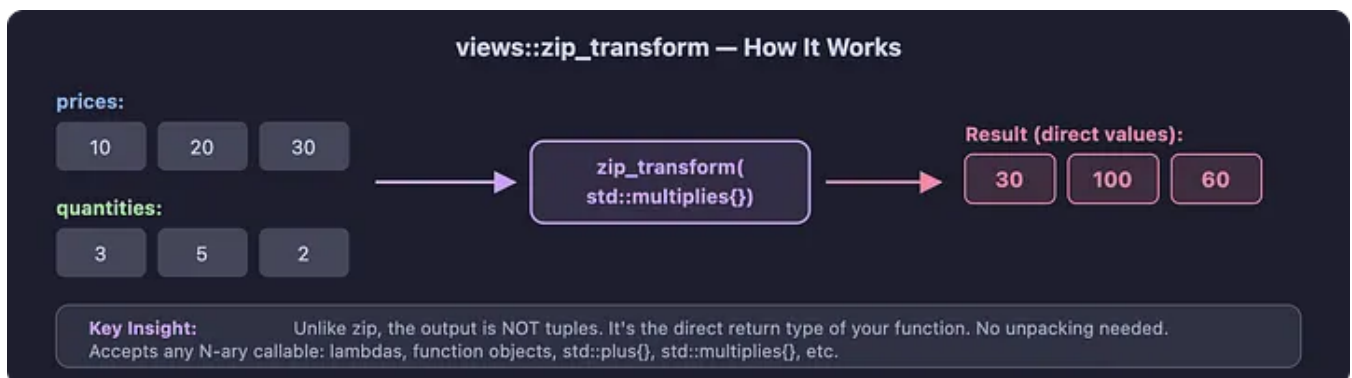
int main() {
    std::vector<std::string> names = {"Alice", "Bob", "Charlie"};
    std::vector<int> scores = {95, 87, 72};

    // Structured bindings, auto-stops at shortest range
    for (auto [name, score] : std::views::zip(names, scores)) {
        std::println("{} scored {}", name, score);
    }
}

```

zip\_transform

**std::views::zip\_transform** is **zip** + **transform** fused into one step. Instead of producing tuples, it takes a callable and applies it across the zipped elements, producing the transformed result directly.



### Key Points:

- Output is NOT tuples — unlike **zip**, the result type is whatever your function returns. No unpacking needed.

- Accepts any N-ary callable: lambdas, function objects, `std::plus{} , std::multiplies{} ,` etc.
- Equivalent to `zip(r1, r2) | transform(apply(f))` but more efficient — avoids constructing intermediate tuples.
- Same shortest-range behavior as `zip`.
- Fully lazy — the function is called only when each element is accessed.

## Pre-C++23: Separate loop with manual indexing

```
std::vector<double> prices      = {10.0, 20.0, 30.0};
std::vector<int>    quantities = {3, 5, 2};

// Allocate result, manual multiply
std::vector<double> totals;
totals.reserve(prices.size());
for (size_t i = 0; i < prices.size(); ++i) {
    totals.push_back(prices[i] * quantities[i]);
}
```

## C++23: One-liner pipeline

```
#include <ranges>
#include <vector>
#include <print>

int main() {
    std::vector<double> prices      = {10.0, 20.0, 30.0};
    std::vector<int>    quantities = {3, 5, 2};

    // Multiply price x quantity — lazy view, no allocation
    auto totals = std::views::zip_transform(
        std::multiplies{},
        prices,
        quantities
    );

    for (double t : totals) {
        std::println("Total: {}", t);    // 30, 100, 60
    }
}
```

```
}  
}
```

## 2. `std::views::enumerate`

`std::views::enumerate` wraps any range and produces `{index, value}` pairs on each iteration. Each element becomes a `std::tuple<difference_type, range_reference_t<R>>` — in practice, you de-structure it as `auto [idx, value]`.

Think of it as C++'s answer to Python's `enumerate()`.



### Key Points:

- Index type is `std::ranges::range_difference_t` (typically `ptrdiff_t`, a *signed* integer) — not `size_t`. This avoids the common signed/unsigned comparison trap.
- Value is a reference, not a copy. If the source range is mutable, you can modify elements through `enumerate`.
- Composable in pipelines — you can *enumerate then filter*, and the original indices are preserved. This is impossible with manual counters.
- Zero-overhead abstraction — the index is just an incrementing counter internally. No lookup, no hashing.

- Usable as both a function call ( `std::views::enumerate(range)` ) and a pipe adaptor ( `range | std::views::enumerate` ).

## Pre-C++23: Two ugly options

```
std::vector<std::string> fruits = {"apple", "banana", "cherry", "date"};

// Option A: index-based – loses range-for benefits
for (size_t i = 0; i < fruits.size(); ++i) {
    std::cout << "[" << i << "]" " " << fruits[i] << "\n";
}

// Option B: manual counter – error-prone, easy to forget ++i
size_t i = 0;
for (const auto& fruit : fruits) {
    std::cout << "[" << i << "]" " " << fruit << "\n";
    ++i; // miss this and you have a bug
}
```

## C++23: Python-style enumerate

```
#include <ranges>
#include <vector>
#include <string>
#include <print>

int main() {
    std::vector<std::string> fruits = {"apple", "banana", "cherry", "date"};

    for (auto [idx, fruit] : std::views::enumerate(fruits)) {
        std::println("{} {} ", idx, fruit);
    }
    // [0] apple
    // [1] banana
    // [2] cherry
    // [3] date
}
```

## Pipeline composition — Pre-C++23 vs C++23

Pre-C++23: Filter then manually re-track original indices

```
std::vector<std::string> fruits = {"apple", "banana", "cherry", "date"};

// Need original index AND a filter? Good luck.
for (size_t i = 0; i < fruits.size(); ++i) {
    if (fruits[i].size() > 5) {
        std::cout << "(" << i << ", " << fruits[i] << ")\n";
    }
}
```

C++23: Enumerate first, then filter — original indices preserved automatically

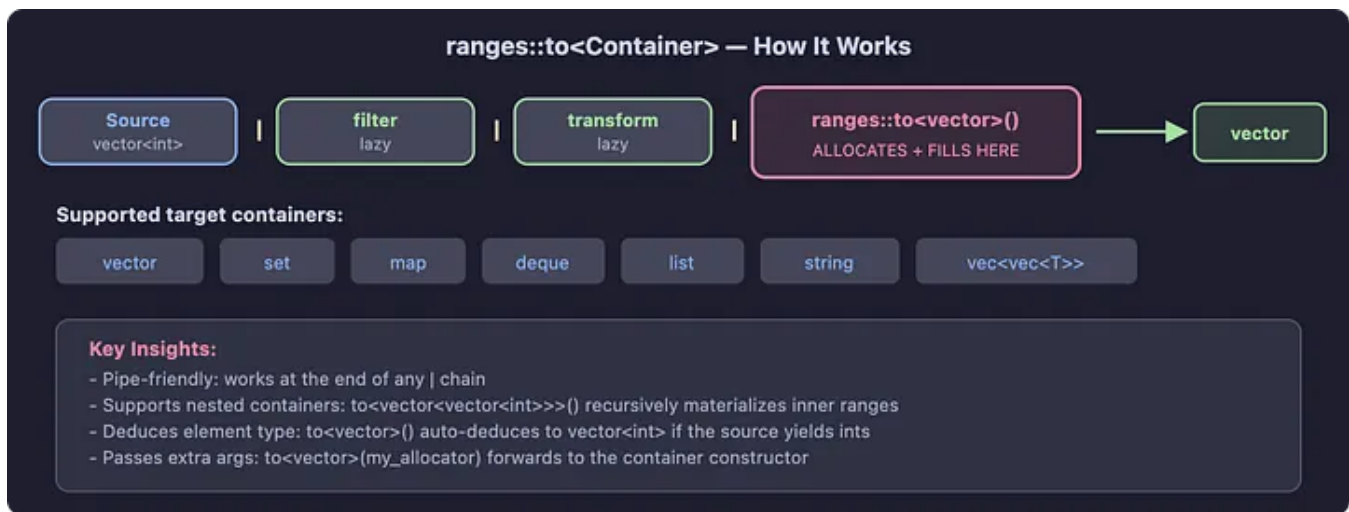
```
auto result = fruits
    | std::views::enumerate
    | std::views::filter([](auto pair) {
        auto [idx, fruit] = pair;
        return fruit.size() > 5;
    });

for (auto [idx, fruit] : result) {
    std::println("({}, {})", idx, fruit);
}
// (1, banana)
// (2, cherry)
```

### 3. `std::ranges::to<Container>` — The Materializer

`std::ranges::to<Container>` is a range adaptor closure that takes any range or view and materializes it into a concrete container. It's the final piece of the pipe — the bridge between lazy evaluation and owning data.





## Key Points:

- **Pipe-friendly** — works as the terminal step in any | chain. This was the #1 missing feature in C++20 ranges.
- Supports nested containers — `to<vector<vector<int>>>>()` recursively materializes inner ranges too. This means `chunk | to<vector<vector<int>>>>()` just works.
- Deduces element type — `to<std::vector>()` (no inner type) auto-deduces `vector<int>` if the source yields `int`s.
- Forwards extra arguments — `to<std::vector>(my_allocator)` passes the allocator to the vector constructor.
- Works with any container that can be constructed from a range or from iterators: `vector`, `set`, `map`, `deque`, `list`, `string`, etc.
- This is the point where lazy meets eager — everything before `to<>()` in the pipeline is lazy; `to<>()` triggers actual allocation and evaluation.

## Pre-C++23: Verbose iterator-pair construction

```
#include <ranges>
#include <vector>
#include <set>

int main() {
    std::vector<int> numbers = {5, 3, 1, 4, 1, 5, 9, 2, 6};
```

```

// Filter even numbers – view is lazy
auto view = numbers
    | std::views::filter([](int n) { return n % 2 == 0; });

// PAIN: Must use iterator-pair constructor
std::vector<int> evens(
    std::ranges::begin(view),
    std::ranges::end(view)
);

// Even worse for sets
std::set<int> unique_sorted(
    std::ranges::begin(numbers),
    std::ranges::end(numbers)
);
}

```

## C++23: Just pipe it

```

#include <ranges>
#include <vector>
#include <set>
#include <print>

int main() {
    std::vector<int> numbers = {5, 3, 1, 4, 1, 5, 9, 2, 6};

    // Pipeline -> vector. Done.
    auto evens = numbers
        | std::views::filter([](int n) { return n % 2 == 0; })
        | std::ranges::to<std::vector>();
    // evens = {4, 2, 6}

    // Pipeline -> set (automatic dedup + sort!)
    auto unique_sorted = numbers
        | std::ranges::to<std::set>();
    // unique_sorted = {1, 2, 3, 4, 5, 6, 9}

    for (int n : evens) std::println("{} ", n);
}

```

## Nested containers — Pre-C++23 vs C++23

Pre-C++23: Nested loop nightmare

```

auto chunked_view = numbers | std::views::chunk(3);

// Must manually iterate and build inner vectors
std::vector<std::vector<int>>> result;
for (auto chunk : chunked_view) {
    std::vector<int> inner(
        std::ranges::begin(chunk),
        std::ranges::end(chunk)
    );
}

```

[Open in app](#) ↗

≡ Medium



A

## C++23: Nested materialization in one pipe

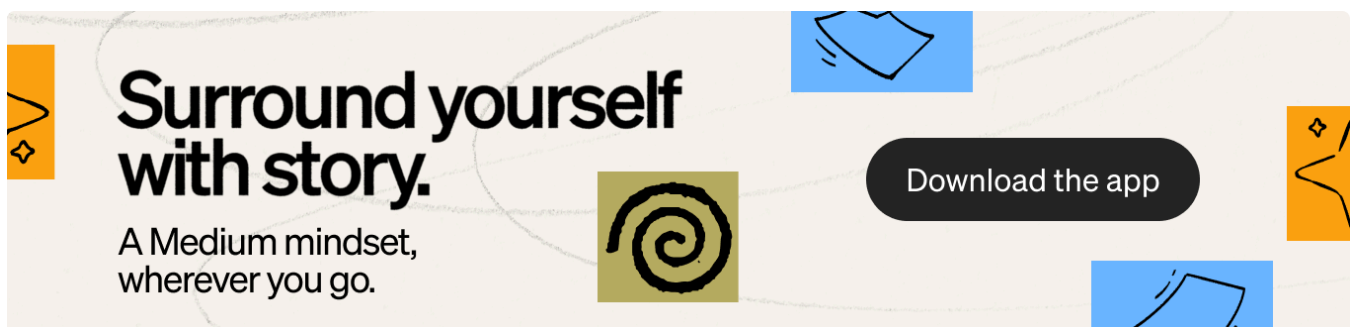
```

auto result = numbers
    | std::views::chunk(3)
    | std::ranges::to<std::vector<std::vector<int>>>>();
// {{5,3,1}, {4,1,5}, {9,2,6}}

```

### 4. chunk, slide, and stride

These three new view adaptors handle windowed/grouped iteration patterns that used to require manual index math.



What Each One Does:

Source: [1, 2, 3, 4, 5, 6, 7]

**chunk(3) — Non-overlapping groups of N**



**Range of ranges.**

Each sub-range has at most N elements.

**slide(3) — Overlapping windows of N**

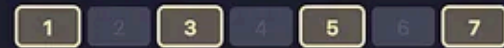


**Shifts by 1 each iteration.**

Every window is exactly N elements.

If range.size() < N, produces empty range (safe!).

**stride(2) — Every Nth element**



Result: {1, 3, 5, 7}

**Flat view (not range-of-ranges).**

Works with any forward range, not just random-access.

stride(1) is identity. stride(N) skips N-1 between.

#### Quick Comparison

| Feature   | Overlapping?          | Output type     | Window size                     |
|-----------|-----------------------|-----------------|---------------------------------|
| chunk(N)  | No                    | range of ranges | Exactly N (last may be smaller) |
| slide(N)  | Yes (by 1)            | range of ranges | Always exactly N                |
| stride(N) | N/A (single elements) | flat range      | N/A (picks every Nth)           |

*All three are lazy. No allocation until consumed or materialized with ranges::to.*

views::chunk(n) — **Pre-C++23 vs C++23**

Pre-C++23: Manual index arithmetic

```
std::vector<int> data = {1, 2, 3, 4, 5, 6, 7};
size_t chunk_size = 3;

for (size_t i = 0; i < data.size(); i += chunk_size) {
    std::cout << "[ ";
    for (size_t j = i; j < std::min(i + chunk_size, data.size()); ++j) {
        std::cout << data[j] << " ";
    }
    std::cout << "]\n";
}
```

C++23: Declarative chunking

```
#include <ranges>
#include <vector>
#include <print>

int main() {
    std::vector<int> data = {1, 2, 3, 4, 5, 6, 7};
```

```

    for (auto group : data | std::views::chunk(3)) {
        std::print("[ ");
        for (int val : group) std::print("{} ", val);
        std::println("]");
    }
    // [ 1 2 3 ]
    // [ 4 5 6 ]
    // [ 7 ]      <- last chunk can be smaller
}

```

views::slide(n) — **Pre-C++23 vs C++23**

Pre-C++23: Off-by-one bug factory

```

std::vector<int> data = {1, 2, 3, 4, 5};
size_t window = 3;

// Easy to get the bounds wrong here
for (size_t i = 0; i <= data.size() - window; ++i) {
    std::cout << "[ ";
    for (size_t j = i; j < i + window; ++j) {
        std::cout << data[j] << " ";
    }
    std::cout << "]\n";
}
// What if data.size() < window? Unsigned underflow. Boom.

```

C++23: Safe sliding windows

```

std::vector<int> data = {1, 2, 3, 4, 5};

for (auto window : data | std::views::slide(3)) {
    std::print("[ ");
    for (int val : window) std::print("{} ", val);
    std::println("]");
}
// [ 1 2 3 ]
// [ 2 3 4 ]
// [ 3 4 5 ]
// Empty range if data.size() < 3. No crash.

```

views::stride(n) — **Pre-C++23 vs C++23**

## Pre-C++23: `i += step` only works for random-access

```
std::vector<int> data = {10, 20, 30, 40, 50, 60, 70};

// Works, but only for random-access containers
for (size_t i = 0; i < data.size(); i += 2) {
    std::cout << data[i] << " ";
}
// 10 30 50 70
// Try this with std::list? Good luck with operator[].
```

## C++23: Works with any forward range, composable in pipelines

```
for (int val : data | std::views::stride(2)) {
    std::print("{} ", val);
}
// 10 30 50 70
// Works with list, deque, any forward range. Composable with other views.
```

## 5. `std::views::join_with`

`std::views::join_with` takes a range of ranges and flattens it into a single range, inserting a delimiter between each inner range. It's the inverse of `split`.

C++20 already had `views::join` (flatten without delimiter). C++23's `join_with` adds the critical ability to inject separators.

### views::join\_with — How It Works

Input: `vector<string>` (range of ranges of char)

Hello

C++23

World

delimiter: " | "



`join_with(" | "sv)`

Output: flat view of char

H e l l o | C + + 2 3 | W o r l d

#### Key Insights:

- Works on ANY range of ranges, not just strings (e.g., `vector<vector<int>>` with an `int` delimiter)
- Delimiter can be a single element (char) or a range (" | "sv) - Lazy: no intermediate string created

### Key Points:

- Works on ANY range of ranges, not just strings. You could flatten `vector<vector<int>>` with an `int` delimiter between sub-vectors.
- Delimiter can be a single value ( ' ' ) or a range ( " | "sv ).
- Lazy evaluation — no intermediate string or container is created. Characters/elements are yielded on demand.
- No delimiter after the last element. `join_with` is smart about this — it only inserts delimiters *between* inner ranges.
- Pairs beautifully with `ranges::to<std::string>()` to materialize the result.
- Inverse of `split`: `split` breaks a range on a delimiter; `join_with` reassembles with a delimiter.

### Pre-C++23: Manual delimiter insertion

```
std::vector<std::string> words = {"Hello", "C++23", "World"};

std::string result;
for (size_t i = 0; i < words.size(); ++i) {
    if (i > 0) result += " | ";    // manual delimiter logic
    result += words[i];
}
std::cout << result << "\n";
// Hello | C++23 | World
```

## C++23: Pipe-friendly join

```
#include <ranges>
#include <vector>
#include <string>
#include <string_view>
#include <print>

int main() {
    std::vector<std::string> words = {"Hello", "C++23", "World"};

    // As a lazy view of chars
    auto joined = words | std::views::join_with(" | "sv);

    for (char c : joined) std::print("{} ", c);
    std::println("");
    // Hello | C++23 | World
}
```

### Converting to `std::string` — Pre-C++23 vs C++23

#### Pre-C++23: Accumulate manually

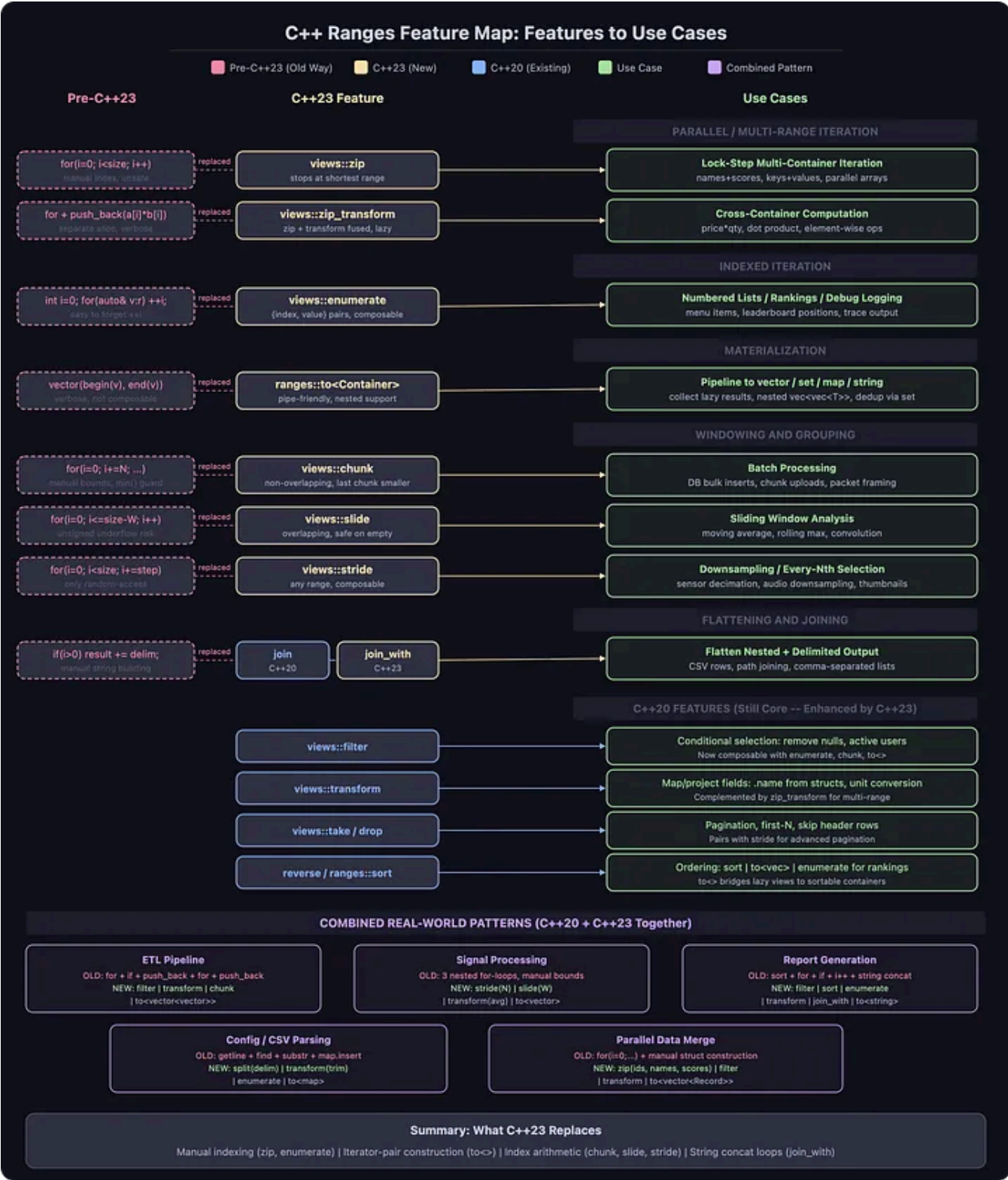
```
std::string sentence;
for (size_t i = 0; i < words.size(); ++i) {
    if (i > 0) sentence += ' ';
    sentence += words[i];
}
```

#### C++23: Pipe to string

```
auto sentence = words
    | std::views::join_with(' ')
    | std::ranges::to<std::string>();
// sentence = "Hello C++23 World"
```



Feature Mapping: Old Way vs New Way to Use Cases



Quick Reference Cheat Sheet

## C++23 Ranges — Quick Reference Cheat Sheet

| Pre-C++23 Way                                                                                                      | C++23 Feature                      | What It Does                                                                                           | Key Insight                                                                                                                                                    | Use Cases                                                                      |
|--------------------------------------------------------------------------------------------------------------------|------------------------------------|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>for(i=0; i&lt;size; i++)</code><br>with <code>a[i]</code> , <code>b[i]</code><br><i>manual index, unsafe</i> | <code>views::zip</code>            | Walks N ranges in lock-step, producing a view of <code>tuple&lt;T&amp;, U&amp;, ...&gt;</code>         | Stops at shortest range. Yields references, not copies. Structured bindings work perfectly. Supports 2+ ranges.                                                | Parallel arrays<br>names + scores<br>keys + values pairing                     |
| <code>for + push_back(a[i] * b[i])</code><br><i>separate alloc, verbose</i>                                        | <code>views::zip_transform</code>  | Zips N ranges and applies a function across them. <code>zip + transform</code> in one step.            | Output is function's return type, NOT tuples. Lazy. No intermediate allocation. Accepts any N-ary callable.                                                    | Element-wise math<br>price * quantity<br>dot product, blending                 |
| <code>int i=0;</code><br><code>for(auto&amp; v:r) ++i;</code><br><i>easy to forget ++i</i>                         | <code>views::enumerate</code>      | Generates (index, value) pairs during iteration. C++ equivalent of Python's <code>enumerate()</code> . | Index is signed <code>ptrdiff_t</code> , not <code>size_t</code> . Preserves original index through filters. Value is a reference, not a copy.                 | Rankings, numbered lists<br>debug logging with index<br>menu items             |
| <code>vector(begin(v), end(v))</code><br><i>verbose, not composable</i>                                            | <code>ranges::to&lt;C&gt;()</code> | Materializes any lazy view into a concrete container. The "collect" at end of pipeline.                | Supports nested containers: <code>to&lt;vec&lt;vec&lt;int&gt;&gt;&gt;&gt;()</code> . Deduces element type automatically. Forwards extra args (allocator, etc). | Pipeline to vector/set/map<br>dedup via set, nested results<br>string building |
| <code>for(i=0; i+=N; ...)</code><br><code>min(i+N, size)</code><br><i>manual bounds guard</i>                      | <code>views::chunk(N)</code>       | Splits range into non-overlapping groups of N. Returns range-of-ranges.                                | Last chunk can be smaller than N. Each sub-range has at most N elements. Lazy. Composable with <code>to&lt;C&gt;</code> , <code>transform</code> .             | Batch processing<br>DB bulk inserts<br>packet framing, chunk uploads           |
| <code>for(i=0; i&lt;size-W; i++)</code><br><i>unsigned underflow risk</i>                                          | <code>views::slide(W)</code>       | Produces overlapping windows of exactly W elements. Shifts by 1 each iteration.                        | Empty range if size < W (safe!). No unsigned underflow possible. Every window always exactly W elements.                                                       | Moving average<br>rolling max, convolution<br>signal smoothing                 |
| <code>for(i=0; i&lt;size; i+=step)</code><br><i>only random-access</i>                                             | <code>views::stride(N)</code>      | Picks every Nth element. Returns a flat range (not range-of-ranges).                                   | Works with ANY forward range, not just random-access. <code>stride(1)</code> = identity view. Composable in pipelines.                                         | Downsampling<br>sensor decimation<br>audio thinning, thumbnails                |
| <code>if(l&gt;0)</code><br><code>result += delim;</code><br><i>manual string concat loop</i>                       | <code>views::join_with</code>      | Flattens range-of-ranges, inserting a delimiter between each inner range.                              | Works on ANY range-of-ranges, not just strings. Delimiter: single value or range. No trailing delimiter. Lazy.                                                 | CSV output, path joining<br>comma-separated lists<br>log formatting            |

All C++23 views are lazy (no allocation until consumed). Compile with `-std=c++23` (GCC/Clang) or `/std:c++latest` (MSVC).

## Compiler Support (as of 2026)

| FEATURE                                                             | GCC | CLANG | MSVC  |
|---------------------------------------------------------------------|-----|-------|-------|
| <code>`zip`</code> / <code>`zip_transform`</code>                   | 13+ | 17+   | 17.6+ |
| <code>`enumerate`</code>                                            | 13+ | 17+   | 17.6+ |
| <code>`ranges::to`</code>                                           | 14+ | 17+   | 17.6+ |
| <code>`chunk`</code> / <code>`slide`</code> / <code>`stride`</code> | 14+ | 17+   | 17.6+ |
| <code>`join_with`</code>                                            | 14+ | 17+   | 17.6+ |

## Final Thoughts

C++23 ranges aren't a new concept — they're the completion of the C++20 promise. Every feature listed here replaces a specific manual-loop pattern:

- Manual index loops across two containers — -> `zip / zip_transform`
- Forgettable `++i` counters ---> `enumerate`
- Verbose iterator-pair constructors — -> `ranges::to`
- Index arithmetic for windows/batches — -> `chunk / slide / stride`
- `if(i>0)` delimiter hacks ---> `join_with`

*“C++20 gave us the tools. C++23 removes the excuses.”*

Write pipelines. Let the compiler do the indexing. Ship cleaner code.

*Found this article helpful? Clap 🙌 and follow me for more C++ and system programming content.*

Programming

Software Development

Cpp

C Plus Plus Language

Cpp23



Follow

## Published in Towards Dev

9.8K followers · Last published 13 hours ago

A publication for sharing projects, ideas, codes, and new theories.



Following ▾